OPSINTEL AI

Evidence-based pipeline investigation

From failed overnight run to traceable next action - powered by Exasol Personal

EXATHON INDIA 2026 / TRACK 04: THE WORK BEHIND THE WORK

Public prototype • MIT licensed • Synthetic data

THE PROBLEM

At 3:07 a.m., the alert says failed. The evidence is scattered.

MANUAL

Evidence lives in separate views

This run expected 100 rows but received 80. The engineer still searches separate run, count, and log views.

OPSINTEL

01 Select the failed run

02 Review cited status, counts, and logs

03 Keep facts separate from candidate explanations

04 System abstains until a human verifies

No time saving is claimed yet. The prepared study measures investigation time and evidence selection.

PRODUCT WORKFLOW

Operational evidence flows through Exasol, FastAPI, and the dashboard

01 **Pipeline runs and logs** →

02 **Exasol SQL analytics** →

03 **FastAPI rules and candidates** →

04 **Dashboard + optional LLM narrative** →

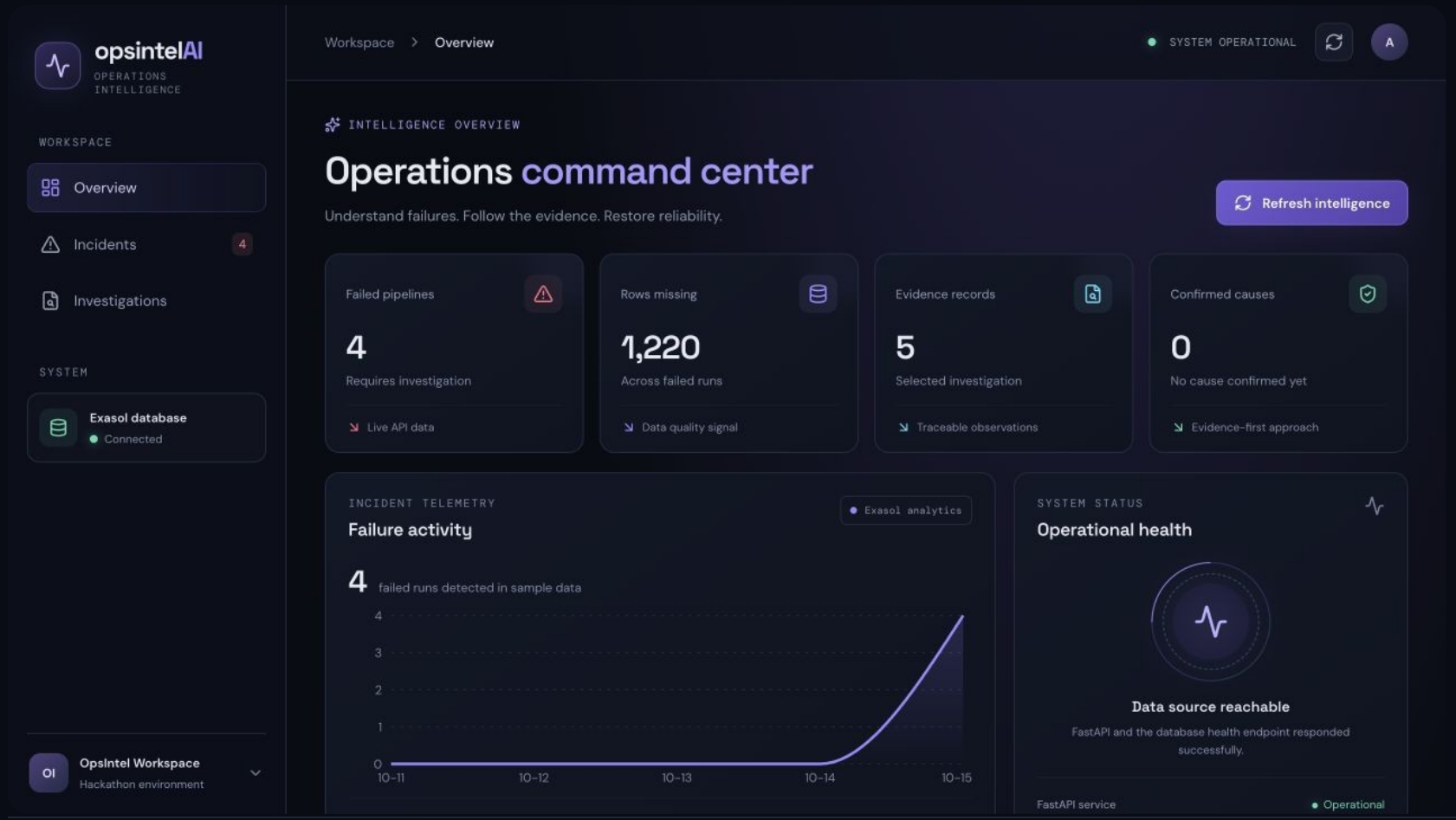
05 **Engineer verifies and confirms**

Safety boundary

Rules produce the category and candidates. The optional LLM only summarizes retrieved evidence. Confirmation and repairs stay human-controlled.

LIVE PRODUCT

The dashboard reads incident telemetry calculated in Exasol



4

failed pipeline runs

1,220

aggregate missing rows in the captured 4-case baseline

✓ Captured baseline: 4 incidents
✓ Current seed: 11 incident scenarios
✓ All values are synthetic

INVESTIGATION EXPERIENCE

Every recommendation points back to source evidence

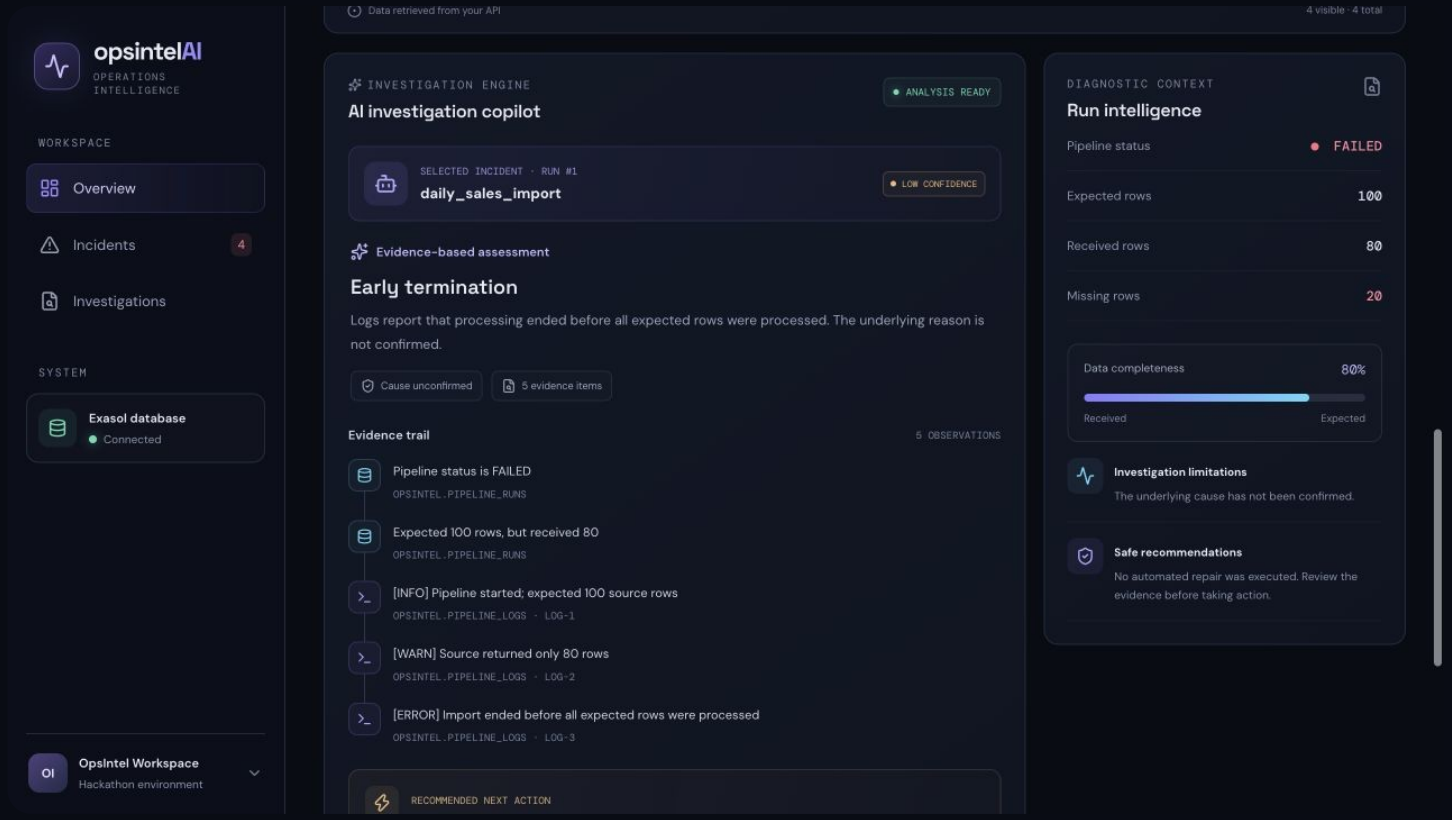
Selected failure vs last success

daily_sales_import

FAILED VS LAST SUCCESS

Failed: 80 / 100 rows (80%)
Last success: 100 / 100 (100%)
Change: -20 rows / -20 pp

EVIDENCE TRAIL
PIPELINE_RUNS: status and row counts
LOG-1: expected 100 source rows
LOG-2: source returned only 80 rows
LOG-3: import ended before completion



CANDIDATES

Source shortfall or early termination
Confidence: low

HUMAN CONFIRMATION

Not yet verified
No automated repair

WHY EXASOL

Exasol computes incident context before FastAPI responds

ANALYTICAL FEATURES

01 Previous run status with LAG

02 Rolling actual-row baseline

03 Rolling failure rate

04 Missing rows and completeness

05 Parallel in-memory SQL execution

QUERY USED BY GET /analytics/trend

```
SELECT RUN_DAY, COUNT(*) AS TOTAL_RUNS,
       SUM(IS_INCIDENT) AS INCIDENT_COUNT,
       ROUND(AVG(COMPLETENESS_PCT), 1) AS AVG_COMPLETE
FROM OPSINTEL.PIPELINE_HEALTH_FEATURES
GROUP BY RUN_DAY;
```

CURRENT SEED · LIVE EXASOL OUTPUT

12 runs · 11 incidents · 64.1% average completeness

GET /analytics/trend reads this view directly. The current seed output was verified against Exasol Personal.

EXASOL FINDS THE SHARED FAILURE

One upstream failure correlates with three downstream incidents

01

Window analytics at scale

LAG, rolling baselines and failure rates compare each run with recent history inside Exasol.

02

Failure cascade

CONNECT BY retains downstream failures within a 120-minute window. Temporal correlation only. Causation not confirmed.

03

Operational evidence layer

FastAPI returns the same SQL-derived metrics and source records that judges can inspect in the dashboard.

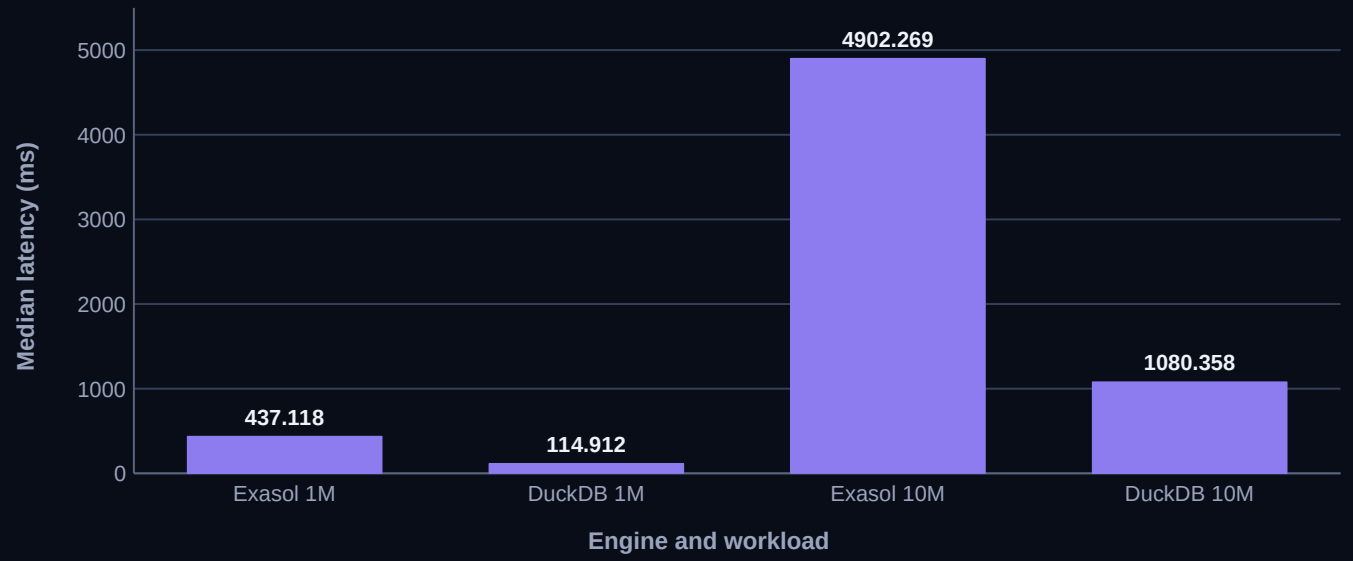
IMPLEMENTED CASCADE QUERY

customer_sync → **daily_reporting** · **customer_orders_sync** · **billing_customer_export**

Live result: customer_sync correlates with 3 later failed runs. Bulk import loads benchmark history; no UDF is implemented.

SAME QUERY, MEASURED HONESTLY

DuckDB was 4.5× faster at 10M rows in this local single-user test



133 / 133

automated tests passed

27 / 27

project-authored holdout

CACHE OFF

Exasol comparison

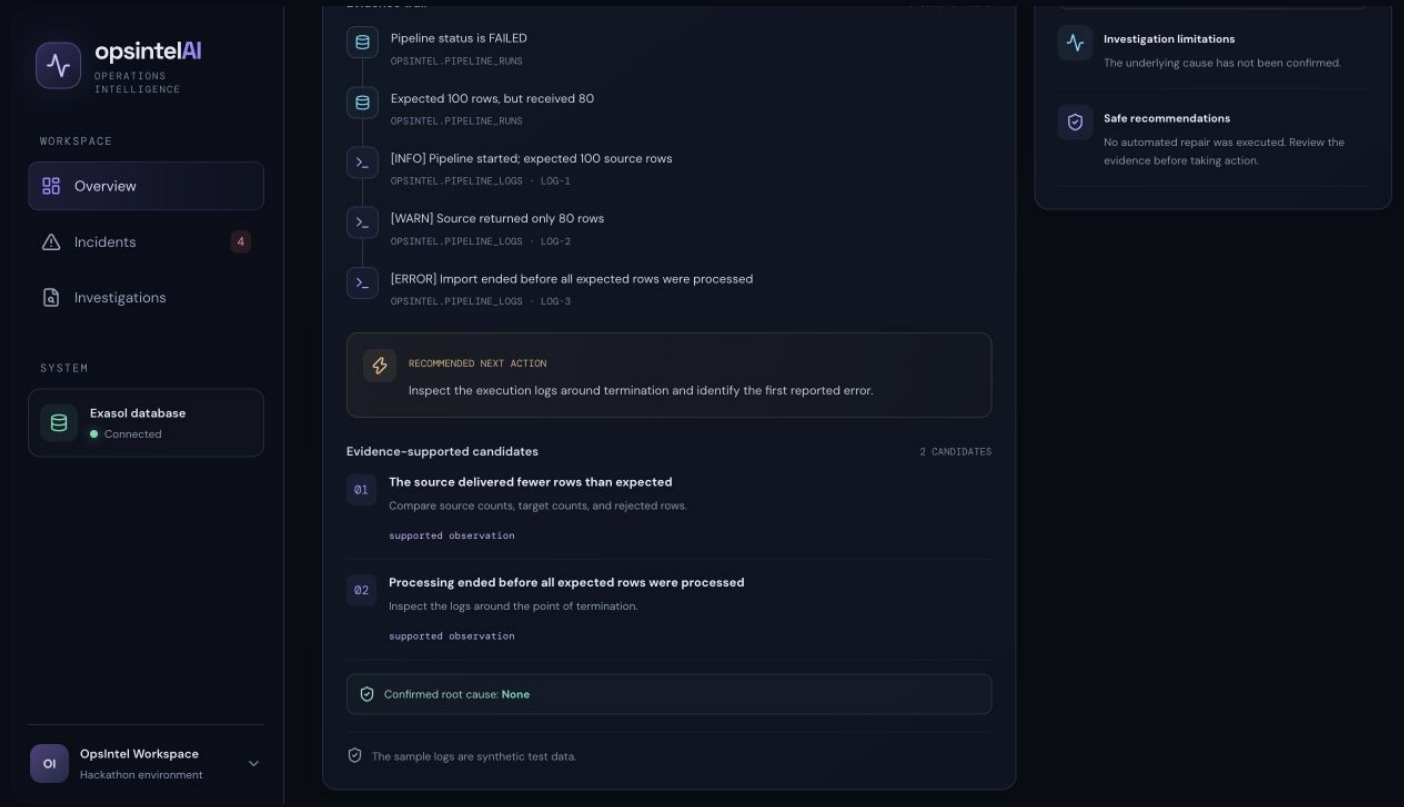
7 RUNS

per engine

Same deterministic query, 7 runs per engine, Exasol query cache OFF. 1M medians: Exasol 437 ms, DuckDB 115 ms. No production or concurrency claim.

TRUST AND SAFETY

The product is designed to abstain before it overclaims



OBSERVATION

Source-backed facts

CANDIDATE

Evidence-supported possibility

CONFIRMATION

Reserved for human verification

INSUFFICIENT EVIDENCE EXAMPLE
Worker exited with code 71. No diagnostic details. Result: unknown. The system says what to check next and cannot confirm a cause or run a repair.

VALIDATION ROADMAP

What is proven, and what must be proven next

PROVEN NOW

01 Live Exasol-backed demo

02 Controlled 1M/10M comparison

03 133 automated tests

04 Clean clone: setup and checks passed

DEFINED, NOT COMPLETED

01 External holdout: 0 reviewer cases

02 Independent LLM quality evaluation

03 Impact study: 0/40 trials (0/4 people)

04 Exasol workflow in a clean environment

PRODUCTION WORK

01 Live Airflow / dbt connectors

02 Authentication and authorization

03 Concurrent workload testing

04 Audited retention and monitoring

Clean-clone checks passed. Exasol workflow, external cases and participant trials remain pending.

OPSINTEL AI

Follow the evidence. Investigate faster. Confirm carefully.

github.com/Anafa-thabassum/opsintel-ai

Live demo: observed facts, candidate causes, human verification